Ignytion io

IGNYTION IO · TECHNICAL LAB NOTE

Find and Prove a FIFO Count Bug

A causal counterexample, corrected RTL, and unbounded safety with managed proof tools

A FIFO must preserve both the number and order of queued bytes. This experiment starts with a broken simultaneous-transfer count update. A solver finds the failure, and the same concrete inputs reproduce it in simulation. The corrected four-entry byte FIFO passes an unbounded safety proof, reachability covers, and an independent scoreboard regression.

SECTION 01

What passed, and what deliberately failed

The **corrected four-entry, eight-bit FIFO** passes its stated single-clock contract. The preserved broken implementation is rejected by formal checking and independent simulation.

CHECK	MEASURED RESULT	STATE
Broken-fixture guard	Occupancy assertion fails at formal step 4	✓ PASS
Corrected safety	Basecase and temporal induction pass	✓ PASS
Reachability	All 12 cover goals reached	✓ PASS
Independent simulation	2,195 cycles · 13,170 checks	✓ PASS
Concrete witness replay	Corrected passes; broken count is 2, expected 1	✓ PASS

The negative rows mean that the **verification guard rejected the intended bug**. They do not mean that the broken RTL passed. Its formal process exits 2; both broken simulation runs exit 1.

Crucible is beta. The local workspace binds the managed SymbiYosys proof bundle and Icarus Verilog. The bundle supplies the actual Yosys/SMT proof dependencies used by SBY.

> **Proof scope**
> This result covers the exact fixed **4 × 8-bit untimed RTL**, after the required initial reset edge. It does not establish physical timing, asynchronous-clock behaviour, liveness, FPGA operation or silicon readiness.

SECTION 02

Specify acceptance at the boundaries

Requests are sampled on a rising edge. Acceptance depends on **pre-edge occupancy**, with full at four bytes and empty at zero.

PRE-EDGE STATE	REQUESTS	ACCEPTED ACTION
Empty	Write and read	Write only
Full	Write and read	Read only
Between empty and full	Write and read	Both; occupancy unchanged
Full	Write only	Rejected; contents retained
Empty	Read only	Rejected; no valid response

The read response is **registered**. An accepted read produces the oldest byte and asserts read-valid after the edge. Otherwise read-valid is low and read data hold their previous value.

Synchronous active-high reset clears both pointers, occupancy, read-valid and read data. It discards queued bytes. Unused memory locations are not reset because they are unobservable until a valid write makes them part of the queue.

There is no fall-through or full-through bypass in this contract. The implementation uses a circular memory; the observer uses a shift queue.

SECTION 03

Keep the environment and model independent

The only input assumption is **reset on the first formal clock edge**. Later resets, requests and every write-data bit remain unconstrained.

PROPERTY GROUP	OBSERVATION
Capacity and occupancy	Level is at most four and equals the model count
Flags	Full and empty agree with model occupancy
Read response	Valid and exact ordered/held data agree
Pointer relation	Circular tail equals head plus model occupancy modulo four
Queued-data relation	Each occupied circular location equals the corresponding model byte

The reference accepts writes and reads from **its own count**. It never uses DUT full/empty flags to decide which operations should occur. A faulty flag cannot make the model silently follow the same error.

Pointer and occupied-memory assertions are **proved relational lemmas** that strengthen induction. They are assertions, not assumptions about DUT correctness. Initially unobservable memory values remain arbitrary.

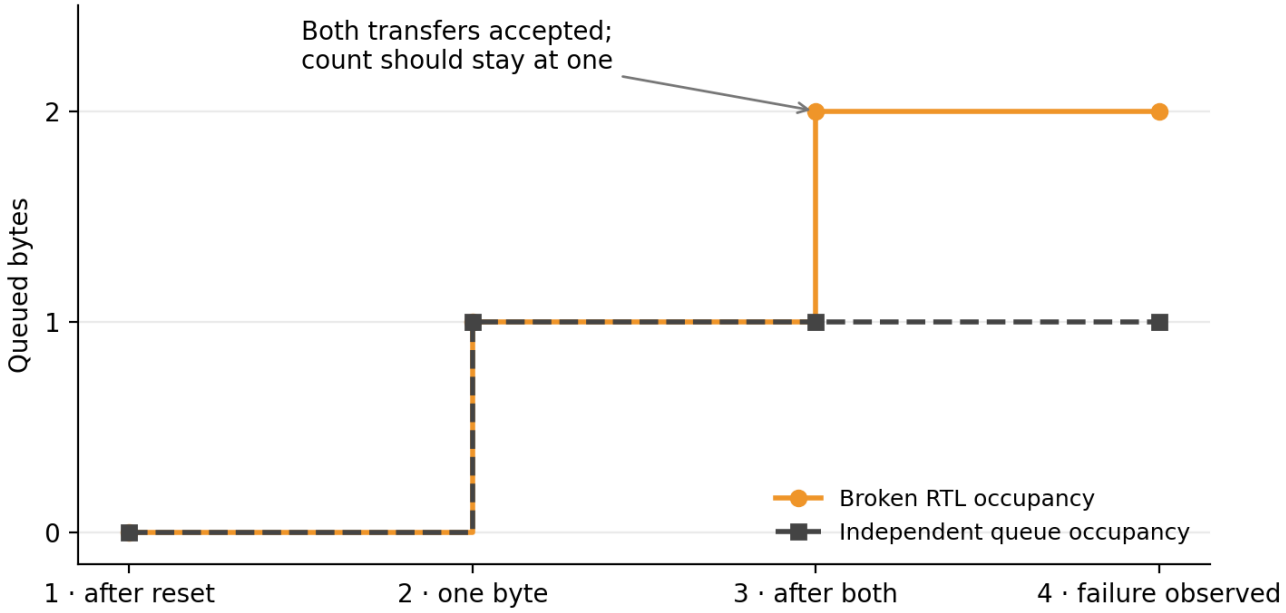
> **Why covers matter**
> Reachability witnesses show that the environment can fill, drain, wrap, transfer simultaneously and reset during activity. A proof that held reset forever would fail these covers.

SECTION 04

Read the actual counterexample

The solver selects a write followed by **simultaneous accepted read and write**. The broken count update gives write priority and increments instead of holding.

ACTUAL SOLVER WITNESS



FORMAL STATE	BROKEN LEVEL	MODEL COUNT
1 · after reset	0	0
2 · one queued byte	1	1
3 · after both transfers	2	1

Figure 1 · Concrete occupancy from the saved VCD. The unconstrained pre-reset state is omitted. The trace first diverges in state 3; SBY reports the sampled occupancy assertion failure at step 4.

The replay uses **only the concrete input sequence**, including the initial reset. It does not force the solver's arbitrary internal initial state into the simulated FIFO.

SECTION 05

Correct the net change in occupancy

The corrected count update uses the **net accepted transfer count**. It preserves the memory and pointer operations and the same verification contract.

ACCEPTED WRITE	ACCEPTED READ	OCCUPANCY CHANGE
0	0	0
1	0	+1
0	1	−1
1	1	0

With both operations accepted, one byte leaves the head and another enters the tail. The queue length stays constant. The broken fixture instead increments whenever a write is accepted, even when the read is accepted too.

The test first observed a genuine **occupancy assertion failure** on the preserved broken implementation. Only then was the corrected RTL introduced. The same formal contract and simulation scoreboard are used for both versions.

```
Formal broken: a_level fails · step 4 · exit 2
Simulation broken: level=2 expected=1 · exit 1
Witness replay broken: level=2 expected=1 · exit 1
```

A compiler error is not this negative result. An initial duplicate assertion-label error was fixed before the solver could examine behaviour; that failed tool setup was never counted as detection of the FIFO bug.

SECTION 06

Distinguish induction from a bounded check

The corrected task runs in **prove mode** with an SMTBMC/Yices engine. Its result includes a passing basecase and successful temporal induction.

```
engine_0 returned pass for basecase
engine_0 returned pass for induction
successful proof by k-induction
DONE (PASS, rc=0)
```

The configured **depth of 16** supports the induction procedure. The result is not limited to observing sixteen test cycles: the induction step establishes safety for all subsequent abstract clock steps under the stated assumptions.

The observer binds each result to the **actual copied RTL and contract files** in the SBY work directory. It rejects stale success logs when source or property bytes have changed. It also requires the correct task mode, process result and engine verdicts.

ACTUAL TOOL IDENTITY	MEASURED VALUE
SBY	v0.63-11-g6424d15
Yosys in proof bundle	0.63+188 · cede13a74
Proof backend	SMTBMC with Yices
Independent simulator	Icarus Verilog 14.0

SECTION 07

Show that useful behaviour is reachable

All **12 cover statements** produce concrete solver witnesses. Their reported steps are abstract formal indices, not physical clock periods or latency benchmarks.

REACHED BEHAVIOUR	FORMAL STEP
Empty without reset	2
Both requests while empty	2
Read request while empty	2
Both operations accepted	3
Reset while occupied	3
Valid read data equal 0xA5	4
Full without reset	6
Both requests while full	6
Write request while full	6
Five accepted writes since reset	7
Five accepted reads since reset	8
Full followed by drain to empty	10

Five accepted operations establish wrap of the corresponding two-bit pointer. The full-to-empty witness must drain without using reset to erase the history marker. These covers supplement the safety proof; they do not add fairness assumptions.

SECTION 08

Replay the witness and test independently

Icarus runs a separate **post-edge, four-state scoreboard**. The model shifts bytes on accepted reads and appends accepted writes; it never consults DUT flags for its decisions.

OBSERVED REGRESSION EVENT	COUNT
Clock cycles	2,195
Output/property comparisons	13,170
Accepted writes / reads	983 / 967
Simultaneous accepted transfers	394
Rejected write / read requests	95 / 168
Reset edges	11

Directed cases exercise empty/full boundaries, concurrent requests, held read data, reset during activity and repeated pointer wrap. A fixed-seed 2,048-cycle random sequence adds arbitrary request/data combinations and later resets.

```
FIFO_SIM_PASS cycles=2195 checks=13170
FIFO_REPLAY_PASS cycles=5 checks=30
```

The VCD input values are cross-checked against the solver's saved constraints before replay. A terminal clock marker is excluded from the five concrete witness states. The **corrected replay passes**; the broken replay fails on the first bad post-edge count.

SECTION 09

Reproduce, inspect and retain the evidence

Use the **Formal_FIFO repository root in Linux/WSL**. The supplied lock restores the managed proof bundle and simulator; project verification uses Python's standard library.

```
igny init
igny workspace sync
bash scripts/run.sh identity
bash scripts/run.sh broken
bash scripts/run.sh prove
bash scripts/run.sh cover
bash scripts/run.sh sim
python3 scripts/verify.py
```

ARTIFACT	WHAT TO CHECK
reports/broken.log	Expected assertion failure, not a tool error
reports/prove.log	Basecase plus induction verdicts
build/cover/PASS	All twelve reached cover names
reports/counterexample.csv	Concrete inputs and first count divergence
reports/simulation.log	Independent positive comparisons
reports/replay_broken.log	Causal count mismatch on replay

The broken stage intentionally records a nonzero SBY result and requires an actual FAIL witness. Raw proof directories and logs are regenerated locally; compact public summaries and exact reviewed whitepapers are retained in the repository.

SECTION 10

Limits and primary source notes

The proof establishes **safety for this exact FIFO configuration and reset/clock model**. The public source and property hashes identify the implementation that was checked.

It does not prove an asynchronous FIFO or CDC crossing, other depths/widths, physical clock/reset quality, timing closure, synthesis mapping equivalence, FPGA/ASIC implementation or silicon behaviour. It contains no fairness assumption or eventual-delivery liveness guarantee.

Reset flushes queued data. Without an accepted read, delivery is not required to progress. Combinational flag behaviour is interpreted at the abstract synchronous interface; no analogue glitch or metastability claim is made.

PRIMARY REFERENCES

YosysHQ SBY · .sby modes, engines and options
`https://yosyshq.readthedocs.io/projects/sby/en/latest/reference.html`

YosysHQ SBY · FIFO verification introduction
`https://yosyshq.readthedocs.io/projects/sby/en/latest/quickstart.html`

Formal FIFO source, properties and reproduction guide
`https://github.com/CRUCIBLE-WORKBENCH/formal-fifo`

All reported experiment results were checked locally on **1 November 2026**. The implementation, independent reference and deliberately broken fixture belong to this demonstration. Tool authors retain credit and their own licences. No endorsement, clean-machine reproduction or manufacturing qualification is implied.